

# **Sri Lanka Institute of Information Technology**



**Human Computer Interaction**

**Lab Report 05**

**IT23624030 - W.M.R.S.K.Wijesinghe**

## Table of Contents

|                                                                        |    |
|------------------------------------------------------------------------|----|
| Activity 1: Compare Flutter, React Native, Kotlin and Swift .....      | 3  |
| 1.1 Individual Strengths and Weaknesses.....                           | 3  |
| 1.2 Comparison Table .....                                             | 4  |
| 1.3 Suitability for FitFlow .....                                      | 4  |
| 1.4 Recommendation .....                                               | 5  |
| Activity 2: Compare Backend, Database, and Authentication Options..... | 6  |
| 2.1 Backend Frameworks .....                                           | 6  |
| 2.2 Database Options .....                                             | 7  |
| 2.3 Authentication & Authorization.....                                | 7  |
| 2.4 Team-Mid-Sized .....                                               | 8  |
| 2.5 Recommendation .....                                               | 8  |
| Activity 3: Technology Comparison Matrix .....                         | 9  |
| 3.1 Consolidated Technology Options Under Evaluation.....              | 9  |
| 3.2 Criteria and Weights .....                                         | 9  |
| 3.3 Frontend Scoring (1–5 scale × weight) .....                        | 10 |
| 3.4 Backend Scoring .....                                              | 10 |
| 3.5 Database Scoring.....                                              | 10 |
| 3.6 Authentication Scoring .....                                       | 11 |
| 3.7 Final Recommended Technology Stack.....                            | 11 |
| 3.8 Supporting Rationale .....                                         | 11 |
| Activity 4: Design a High-Level Architecture .....                     | 12 |
| 4.1 Key Components.....                                                | 12 |
| 4.2 Critical data flows .....                                          | 13 |
| 4.3 Security, Scalability, and Integration Considerations .....        | 14 |
| 4.4 Architecture Decision Record (ADR) .....                           | 15 |
| Activity 5: GitHub Repository .....                                    | 16 |

## Activity 1: Compare Flutter, React Native, Kotlin and Swift

### 1.1 Individual Strengths and Weaknesses

| Technology           | Strengths                                                                                                                                                                                                                                                                                                | Weaknesses                                                                                                                                                                                                                                                             |
|----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Flutter              | <ul style="list-style-type: none"><li>• Single codebase for iOS, Android &amp; Web</li><li>• Excellent for pixel perfect, highly custom UIs</li><li>• Hot Reload speeds up development</li><li>• Strong Google backing</li></ul>                                                                         | <ul style="list-style-type: none"><li>• Larger application size</li><li>• Dart is less common, increasing hiring/training time</li><li>• Native feeling UI may require additional work</li></ul>                                                                       |
| React Native         | <ul style="list-style-type: none"><li>• Enormous npm ecosystem</li><li>• Leverages existing JS/TS web development knowledge</li><li>• Almost native performance for most UIs</li><li>• Large community and well integrated with Firebase</li><li>• Good ecosystem via JS libraries and bridges</li></ul> | <ul style="list-style-type: none"><li>• May be slower with heavy graphics/animations</li><li>• Certain device functions need native modules</li><li>• Native bridges may complicate things</li><li>• Version changes may pose compatibility problems</li></ul>         |
| Kotlin Multiplatform | <ul style="list-style-type: none"><li>• Exposes business logic but retains native UI</li><li>• Good performance and native UI style</li><li>• Ideal for Android centric organizations</li><li>• Provides platform specific implementations where necessary</li></ul>                                     | <ul style="list-style-type: none"><li>• Ecosystem not yet fully grown</li><li>• UI mostly has to be built separately for both Android and iOS</li><li>• Fewer web features</li><li>• Small community relative to Flutter and RN</li></ul>                              |
| Swift / SwiftUI      | <ul style="list-style-type: none"><li>• Best Performance with iOS platform integration</li><li>• Access to the most up to date Apple APIs</li><li>• Good support for ARKit, HealthKit and Core ML</li><li>• Polished native iOS interface</li></ul>                                                      | <ul style="list-style-type: none"><li>• Applicable only to iOS devices</li><li>• Needs separate code for Android and Web platforms</li><li>• Expensive to develop and maintain cross platform apps</li><li>• Cannot be used alone for cross platform support</li></ul> |

## 1.2 Comparison Table

| Criteria           | Flutter               | React Native                                                | Kotlin Multiplatform | Swift/SwiftUI         |
|--------------------|-----------------------|-------------------------------------------------------------|----------------------|-----------------------|
| Development speed  | High                  | High                                                        | Medium               | Low                   |
| Code reusability   | Very high             | Very high                                                   | Medium               | None across platforms |
| Performance        | Near native           | Good, occasional bridge overhead for heavy animation        | Native               | Native                |
| Ecosystem support  | Strong, growing fast  | Very strong                                                 | Growing, smaller     | Very strong           |
| Learning curve     | Medium                | Low                                                         | Medium High          | Medium High           |
| Web compatibility  | Good                  | Good                                                        | Limited/experimental | None                  |
| AI/ML integration  | Good                  | Good                                                        | Good                 | Excellent             |
| Real-time features | Good                  | Excellent                                                   | Good                 | Excellent             |
| Maintenance cost   | Medium                | Medium                                                      | Medium High          | High                  |
| Security           | Good, compiled binary | Good, but JS bridge can be an attack surface if not secured | Strong               | Strong                |

## 1.3 Suitability for FitFlow

FitFlow requires one team to deploy quickly on iOS, Android, and Web, with smooth animations, real time social features, camera based nutrition logging, and robust health data security.

- Swift/SwiftUI solution is immediately ruled out - it will fail to support Android or Web deployment without an entirely separate build, which conflicts with the "seamless iOS/Android/web" requirement.
- Kotlin Multiplatform provides native performance but does not save UI development effort and poorly supports Web deployment - not suitable for a mid-sized team looking to get to market fast.
- Flutter provides the best technical solution but has smaller talent pool and may need custom work for true "native" experience across platforms.
- React Native provides the best compromise: a single JavaScript/TypeScript codebase for iOS + Android + Web (with React Native Web), well developed library base for real time/social features (Firebase, WebSockets), larger talent pool, and good AI/ML integrations (TensorFlow.js, ML Kit/Core ML bridges).

## 1.4 Recommendation

- **Recommended:** React Native with selective use of native modules for performance or AI related tasks. This mixed solution provides FitFlow with the speed/cost efficiency of a single codebase approach, while providing the near native performance required for specific features.
- **Reasoning:** The fastest way to a consistent experience on iOS, Android, and Web largest pool of developers for mid sized teams well developed library base for real time and social features and demonstrated ability to provide fit for fitness app UI animations as reported in FitFlow case study.

## Activity 2: Compare Backend, Database, and Authentication Options

### 2.1 Backend Frameworks

| Backend Framework | Strengths                                                                                                                                                                                                                                                                                                                                                                                                | Weaknesses                                                                                                                                                                                                                                                         |
|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Node.js / NestJS  | <ul style="list-style-type: none"><li>• Same JavaScript/ TypeScript language as React Native</li><li>• Better team productivity for full stack development</li><li>• NestJS gives a structured and modular approach</li><li>• Very large npm package ecosystem</li><li>• Great real time capabilities using Socket.io &amp; WebSocket</li><li>• Maturity with Firebase &amp; Auth0 integration</li></ul> | <ul style="list-style-type: none"><li>• The single threaded event loop must handle CPU intensive operations judiciously.</li><li>• CPU intensive machine learning inferences must be handled by a different AI microservice.</li></ul>                             |
| Python / FastAPI  | <ul style="list-style-type: none"><li>• Perfect platform for building AI/ML models</li><li>• Supports Tensorflow, PyTorch, and OpenCV</li><li>• Great for vision based applications such as identifying foods</li><li>• Asynchronous performance is fast</li><li>• Auto generates APIs</li><li>• Ideal for AI microservices</li></ul>                                                                    | <ul style="list-style-type: none"><li>• Use of different language compared to the frontend, adding another context switch</li><li>• Real time/WebSocket infrastructure is not as developed as Node.js</li><li>• Need for additional scalability settings</li></ul> |
| Go                | <ul style="list-style-type: none"><li>• Superior performance and concurrency</li><li>• High efficiency</li><li>• Ideal for real time, high throughput services</li><li>• Effective for scalable back end systems</li></ul>                                                                                                                                                                               | <ul style="list-style-type: none"><li>• Smaller pool of developers</li><li>• Can be harder to learn</li><li>• Slow development for CRUD heavy applications</li><li>• Fewer native AI/ML libraries available than Python</li></ul>                                  |

## 2.2 Database Options

| Database   | Scalability          | Query Performance                                             | Health Data Handling                                                                        |
|------------|----------------------|---------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| PostgreSQL | High                 | Excellent for complex relational queries, joins, transactions | Strong fit ACID compliance suits sensitive health records, supports row level security      |
| MongoDB    | High                 | Fast for document style, flexible schema                      | Good for unstructured/varied data, needs careful schema design for compliance               |
| Firebase   | Auto-scales, managed | Good for simple queries, real time sync built in              | Usable but less granular control over data residency/compliance than self managed DBs       |
| DynamoDB   | Very high            | Excellent at scale, but limited complex querying              | Good if paired with strict IAM policies, less flexible for relational health data reporting |

## 2.3 Authentication & Authorization

| Option        | Security/Compliance                                | Real-time Fit                             | Cost                                    | Maintainability                    |
|---------------|----------------------------------------------------|-------------------------------------------|-----------------------------------------|------------------------------------|
| Firebase Auth | Good, GDPR - capable with configuration            | Excellent                                 | Free tier generous, scales with usage   | Very low - fully managed           |
| AWS Cognito   | Strong, HIPAA - eligible under AWS BAA             | Good, pairs well with DynamoDB/ AWS stack | Pay as you go, can get complex to price | Medium - more configuration needed |
| Auth0         | Very strong, enterprise - grade compliance tooling | Good                                      | Higher cost at scale                    | Low - managed, well documented     |
| Supabase Auth | Good, open source                                  | Good                                      | Low cost, free self-host option         | Medium                             |

## 2.4 Team-Mid-Sized

- FitFlow handles health data, so HIPAA/GDPR friendly options matter.
- Real time social features favor Firebase or a Node.js/Socket.io setup.
- AI/computer vision is best served by a Python/FastAPI microservice, kept separate from the main API.
- A mid sized team benefits most from managed services to reduce DevOps overhead versus a large platform team needed for raw AWS DynamoDB + Cognito configuration.

## 2.5 Recommendation

- **Backend:** Node.js + Express for the core API service and real time social/notification service, along with a separate Python based FastAPI microservice for computer vision based nutrition logging and personalization models.
- **Database:** PostgreSQL as the main relational database for storing structured sensitive data and Firebase/Firestore for real time social features.
- **Authentication:** Firebase Auth as the solution for tight integration with real time services and quick setup, with added security measures in place for HIPPA/GDPR compliance on the data level .

**Justification:** It satisfies the requirements of mixed approach, privacy sensitive nature of the application discussed in the case study, allows a mid sized team to avoid DevOps problems, provides isolation and scalability of AI workloads, and ensures good real-time capabilities for social features with PostgreSQL ensuring data integrity.

## Activity 3: Technology Comparison Matrix

### 3.1 Consolidated Technology Options Under Evaluation

**Frontend:** Flutter, React Native, Kotlin Multiplatform, Swift/SwiftUI

**Backend:** Node.js/NestJS, Python/FastAPI, Go

**Database:** PostgreSQL, MongoDB, Firebase, DynamoDB

**Authentication:** Firebase Auth, AWS Cognito, Auth0, Supabase Auth

### 3.2 Criteria and Weights

FitFlow's priorities from the case study: fast time to market for a struggling startup, cross platform reach, AI powered personalization, real time social features, strong security for health data (GDPR/CCPA), and a maintainable cost structure for a mid sized team.

| Criterion           | Weight | Rationale                                                                  |
|---------------------|--------|----------------------------------------------------------------------------|
| Performance         | 15%    | Fitness tracking, animations, and camera based logging need responsiveness |
| Scalability         | 10%    | Must support growth after the 68% drop off is fixed and retention improves |
| Development Speed   | 20%    | Startup under pressure to reverse declining ratings quickly                |
| Security/Compliance | 20%    | Health data GDPR/CCPA compliance is non negotiable                         |
| Cost                | 10%    | Mid sized team, limited budget                                             |
| AI/ML Support       | 15%    | Core differentiator: AI workout engine, computer vision nutrition logging  |
| Maintainability     | 10%    | Small team must sustain the product long term                              |
| Total               | 100%   |                                                                            |

### 3.3 Frontend Scoring (1–5 scale × weight)

| Criteria (Weight)     | Flutter  | React Native | Kotlin Multiplatform | Swift/SwiftUI |
|-----------------------|----------|--------------|----------------------|---------------|
| Performance (15%)     | 4 (0.60) | 4 (0.60)     | 5 (0.75)             | 5 (0.75)      |
| Scalability (10%)     | 4 (0.40) | 4 (0.40)     | 4 (0.40)             | 3 (0.30)      |
| Dev Speed (20%)       | 4 (0.80) | 5 (1.00)     | 2 (0.40)             | 1 (0.20)      |
| Security (20%)        | 4 (0.80) | 4 (0.80)     | 4 (0.80)             | 5 (1.00)      |
| Cost (10%)            | 4 (0.40) | 5 (0.50)     | 3 (0.30)             | 1 (0.10)      |
| AI/ML (15%)           | 3 (0.45) | 4 (0.60)     | 4 (0.60)             | 5 (0.75)      |
| Maintainability (10%) | 4 (0.40) | 4 (0.40)     | 3 (0.30)             | 2 (0.20)      |
| Total                 | 3.85     | 4.30         | 3.55                 | 3.30          |

### 3.4 Backend Scoring

| Criteria (Weight)     | Node.js/NextJS | Python/FastAPI | Go       |
|-----------------------|----------------|----------------|----------|
| Performance (15%)     | 4 (0.60)       | 4 (0.60)       | 5 (0.75) |
| Scalability (10%)     | 4 (0.40)       | 4 (0.40)       | 5 (0.50) |
| Dev Speed (20%)       | 5 (1.00)       | 4 (0.80)       | 3 (0.60) |
| Security (20%)        | 4 (0.80)       | 4 (0.80)       | 4 (0.80) |
| Cost (10%)            | 4 (0.40)       | 4 (0.40)       | 4 (0.40) |
| AI/ML (15%)           | 3 (0.45)       | 5 (0.75)       | 2 (0.30) |
| Maintainability (10%) | 4 (0.40)       | 4 (0.40)       | 3 (0.30) |
| Total                 | 4.05           | 4.15           | 3.65     |

### 3.5 Database Scoring

| Criteria (Weight)     | PostgreSQL | MongoDB  | Firebase | DynamoDB |
|-----------------------|------------|----------|----------|----------|
| Performance (15%)     | 5 (0.75)   | 4 (0.60) | 4 (0.60) | 5 (0.75) |
| Scalability (10%)     | 4 (0.40)   | 4 (0.40) | 5 (0.50) | 5 (0.50) |
| Dev Speed (20%)       | 4 (0.80)   | 4 (0.80) | 5 (1.00) | 3 (0.60) |
| Security (20%)        | 5 (1.00)   | 4 (0.80) | 3 (0.60) | 4 (0.80) |
| Cost (10%)            | 4 (0.40)   | 4 (0.40) | 4 (0.40) | 3 (0.30) |
| AI/ML (15%)           | 3 (0.45)   | 4 (0.60) | 3 (0.45) | 3 (0.45) |
| Maintainability (10%) | 4 (0.40)   | 3 (0.30) | 5 (0.50) | 3 (0.30) |
| Total                 | 4.20       | 3.90     | 4.05     | 3.70     |

### 3.6 Authentication Scoring

| Criteria (Weight)         | Firebase Auth | AWS Cognito | Auth0    | Supabase Auth |
|---------------------------|---------------|-------------|----------|---------------|
| Security/Compliance (20%) | 4 (0.80)      | 5 (1.00)    | 5 (1.00) | 4 (0.80)      |
| Real time Fit (15%)       | 5 (0.75)      | 3 (0.45)    | 3 (0.45) | 4 (0.60)      |
| Cost (10%)                | 5 (0.50)      | 3 (0.30)    | 2 (0.20) | 5 (0.50)      |
| Maintainability (10%)     | 5 (0.50)      | 3 (0.30)    | 4 (0.40) | 4 (0.40)      |
| Dev Speed (20%)           | 5 (1.00)      | 3 (0.60)    | 4 (0.80) | 4 (0.80)      |
| Total                     | 4.20          | 2.65        | 2.85     | 3.10          |

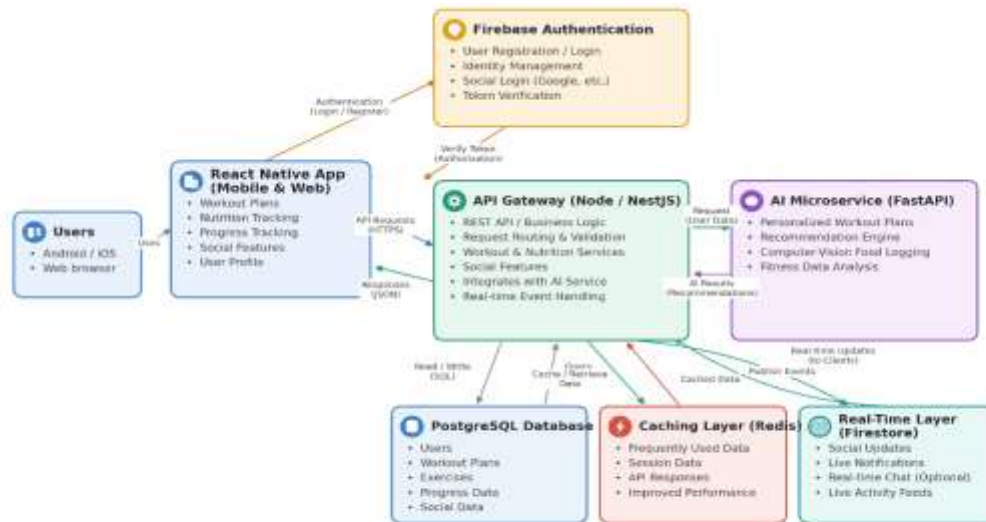
### 3.7 Final Recommended Technology Stack

| Layer                     | Selected Technology | Score      |
|---------------------------|---------------------|------------|
| Frontend                  | React Native        | 4.30 / 5.0 |
| Backend (Main API)        | Node.js / NestJS    | 4.05 / 5.0 |
| Backend (AI Microservice) | Python / FastAPI    | 4.15 / 5.0 |
| Database (Primary)        | PostgreSQL          | 4.20 / 5.0 |
| Database (Real-time)      | Firebase Firestore  | 4.05 / 5.0 |
| Authentication            | Firebase Auth       | 3.55 / 5.0 |

### 3.8 Supporting Rationale

This stack ranks highest among all others in FitFlow's priorities list, it offers the fastest time to market due to the fast paced development of FitFlow in order to recover from a 3.8 star rating slump, high level of security and compliance concerning healthcare data , full fledged AI / ML capabilities in the form of a separate FastAPI microservice dedicated to computer vision nutrition logging, low maintenance overhead, which is suitable for a mid size company, predictable costs of infrastructure. Moreover, this stack matches the stack used by the FitFlow team in the case study.

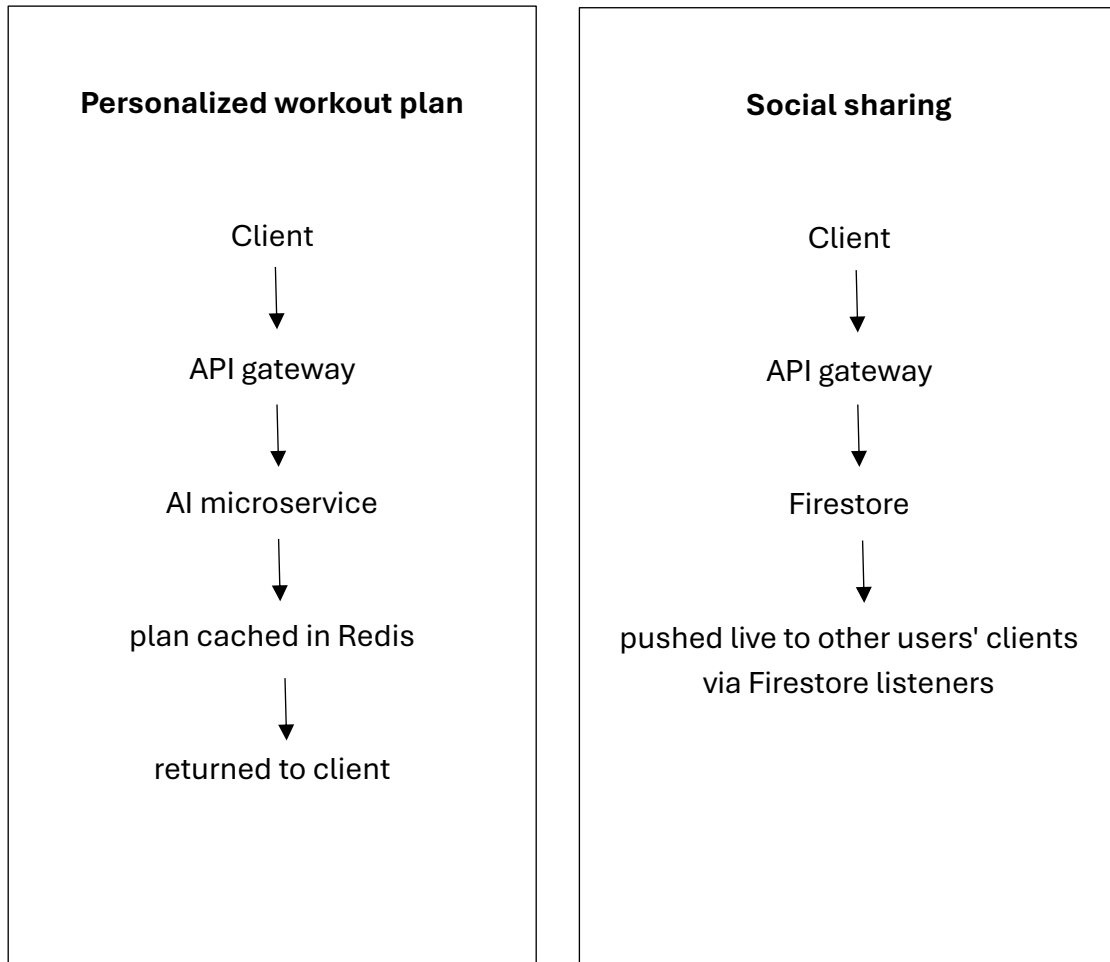
## Activity 4: Design a High-Level Architecture

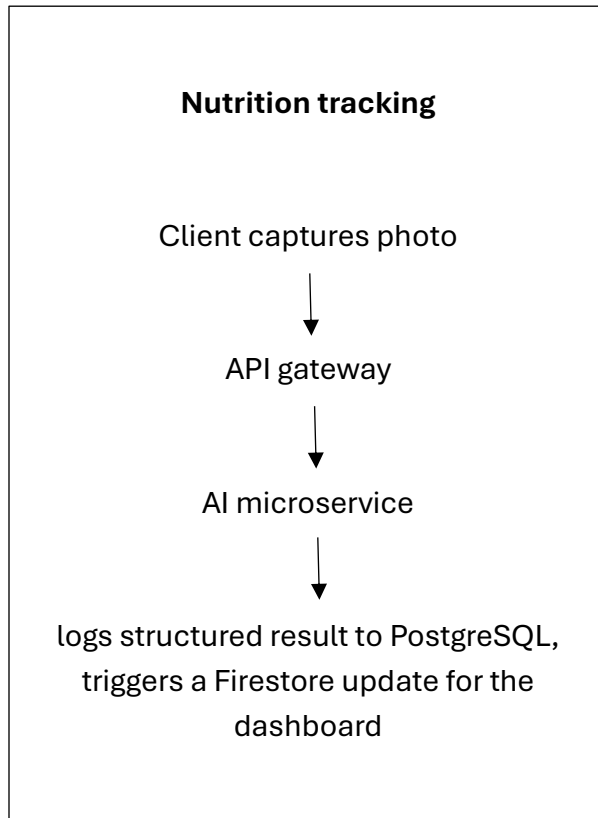


### 4.1 Key Components

- **React Native app (client):** single codebase for iOS, Android, and Web, calling the API gateway for data and Firebase Auth for login.
- **API gateway (Node.js/NestJS):** handles core CRUD, orchestrates requests to the AI microservice, and reads/writes PostgreSQL, Firestore, and Redis.
- **Firebase Auth:** handles login, session tokens, and role based access, the client authenticates directly against it, then attaches tokens to API gateway calls.
- **AI microservice (Python/FastAPI):** serves the AI workout engine and camera based nutrition/computer vision inference, called by the API gateway rather than directly by the client, and caches frequent inference results in Redis.
- **PostgreSQL:** system of record for users, health metrics, and workout history.
- **Firestore:** powers real time features (social feed, challenge updates, live notifications).
- **Redis cache:** reduces repeated load on PostgreSQL/AI microservice for frequently requested data.

## 4.2 Critical data flows:





### 4.3 Security, Scalability, and Integration Considerations

- **Security:** All traffic over HTTPS/TLS , Firebase Auth issues short lived JWTs verified at the API gateway , PostgreSQL enforces row level security on health data , health sensitive fields encrypted at rest , GDPR/CCPA compliant consent and deletion flows built into the API layer.
- **Scalability:** API gateway and AI microservice are stateless and horizontally scalable behind a load balancer , PostgreSQL scales via read replicas , Firestore auto scales for real time load , Redis absorbs read heavy spikes.
- **Integration:** AI microservice is decoupled from the main API so ML models can be updated/deployed independently without redeploying the whole backend , Firebase Auth and Firestore share the same SDK ecosystem, simplifying real time + auth integration on the client.

## 4.4 Architecture Decision Record (ADR)

**Title:** Technology Stack and Service Boundaries for FitFlow Redesign

**Status:** Accepted

**Context:** The rating of FitFlow was falling because of general workout plan, dull nutrition recording, and absence of social functions. The new design will require AI driven workouts, camera supported nutrition recording, real time social functions, and should work flawlessly on iOS, Android, and Web with GDPR/CCPA data protection for mid size team with time limitations.

**Decision:** Adopt React Native for the client , split the backend into a Node.js/NestJS API gateway and a separate Python/FastAPI AI microservice , use PostgreSQL as the system of record, Firestore for real time social/notification data, Redis for caching, and Firebase Auth for authentication.

### Consequences:

- **Positive:** Single frontend code base saves on development effort, the decoupling of the AI microservice enables the data science team to work separately, PostgreSQL provides excellent guarantees regarding health data, managed services minimize DevOps effort for the medium sized team.
- **Negative:** Having two backend programming languages (Node.js and Python) leads to some extra complexity, using both PostgreSQL and Firestore requires handling data consistency between two databases.
- **Mitigation:** Maintain a small and clean API of the AI microservice, utilize Firestore only for truly real time and ephemeral data.

## **Activity 5: GitHub Repository**

Git link:- <https://github.com/IT23624030/fitflow-redesign.git>